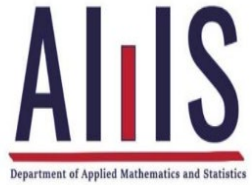


Institute of Technology of Cambodia



Department of Applied Mathematics and Statistics

Group Project: 06

Course Subject: Data Structure and Programming II

Topic: Student Information Management System

Name	ID	Score
1. HIN Panha	e20241363	
2. HOR Soue Hak	e20240263	
3. KUN Virak	e20240486	

Course: Dr. PHAUK Sökkhey

TD: Ms. UANN Sreyvi

Academic year: 2025-2026

Introduction

Managing student information efficiently is an important task in educational environments, especially when dealing with multiple student records. A Student Management System provides a simple way to store, organize, search, update, and manage student information in a structured manner.

In this project, we developed a **Student Management System using C++**, applying fundamental programming concepts such as **structures, pointers, linked lists, queues, file handling, searching, and sorting**. The system uses a queue implemented with a linked list to manage student records dynamically, while a CSV file is used to store information permanently so that previously saved records can be loaded when the program starts.

The system provides several main functions, including **adding, removing, searching, updating, sorting, and displaying student records**. Student information such as ID, name, age, department, phone number, and email can be managed through the system. The project also applies **Bubble Sort** to organize students alphabetically by name and provides a menu-based interface for users to interact with the system.

Overall, this project demonstrates how data structures and file handling can be combined to create a practical and organized student information management application.

Table of Contents

I. Libraries and Structure Definition	1
II. Queue Initialization.....	2
III. Save Data to File	2
IV. Load Data to File	3
V. Add Student	6
VI. Remove Student	8
VII. Sort Student Records	10
VIII. Display Student.....	12
IX. Search Student.....	14
X. Update Student Information.....	16
XI. Menu Function.....	21
XII. Main Function.....	22

I. Libraries and Structure Definition

Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

struct Student {
    string name;
    string id;
    int age;
    string gender;
    string department;
    string phone;
    string birth;
    string email;
    int year;
    string major;
    string province;

    Student* next;
};
```

- Import required libraries.
- Define the Student structure.
- Store student information such as ID, name, age, department, phone number, and email.

- Use a pointer next to create a linked list.

II. Queue Initialization

 Code

```
Student* front = NULL;  
Student* rear = NULL;
```

- Initialize the queue.
- front points to the first student.
- rear points to the last student.

III. Save Data to File

 Code

```
//Save data to CSV file  
  
void SaveFile () {  
    ofstream file("Students_Information.csv");  
  
    file <<  
    "ID,Name,Age,Gender,Department,Major,Year,Province,Phone,Birth  
    ,Email" << endl;  
  
    Student* current = front;  
    while (current != NULL) {
```

```

        file << current -> id << ","
            << current -> name << ","
            << current -> age << ","
            << current -> gender << ","
            << current -> department << ","
            << current -> major << ","
            << current -> year << ","
            << current -> province << ","
            << current -> phone << ","
            << current -> birth << ","
            << current -> email << endl;

        current = current -> next;
    }
    file.close();
}

```

- Save student records into a CSV file.
- Allow data persistence after the program closes.
- Write all information from the queue to Students_Information.csv.

IV. Load Data to File



```

//Turn data from CSV file into queue

void DataIntoQueue (){

    ifstream file("Students_Information.csv");

    if (!file){

        return;

        string skip;

        getline (file, skip);

        string id, name, ageText, gender, department,major,
        yearText, province, phone, birth, email;

        while (getline (file, id, ',') &&
                getline (file, name, ',') &&
                getline (file, ageText, ',') &&
                getline (file, gender, ',') &&
                getline (file, department, ',') &&
                getline (file, major, ',') &&
                getline (file, yearText, ',') &&
                getline (file, province, ',') &&
                getline (file, phone, ',') &&
                getline (file, birth, ',') &&
                getline (file, email)){

            Student* newStudent = new Student();

            //store whole data into queue

            newStudent -> id = id;

            newStudent -> name = name;

```

```

newStudent -> age = stoi(ageText);
newStudent -> gender = gender;
newStudent -> department = department;
newStudent -> major = major;
newStudent -> year = stoi(yearText);
newStudent -> province = province;
newStudent -> phone = phone;
newStudent -> birth = birth;
newStudent -> email = email;
newStudent -> next = NULL;

if (rear == NULL){
    front = rear = newStudent;
} else {
    rear -> next = newStudent;
    rear = newStudent;
}
}

file.close();
}

```

- Read student data from the CSV file.
- Convert file data into queue nodes.
- Restore previously saved records when the program starts.

V. Add Student



Code

```
//Add student
void addStudent () {
    Student* newStudent = new Student;
    Student* idStudent = front;

    cout << "\nEnter Student ID: "; cin >> newStudent->id;

    while (idStudent != NULL) {
        if (newStudent->id == idStudent->id) {
            cout << "\nThis student's ID is already registered" << endl;
            return;
        }
        idStudent = idStudent->next;
    }

    cin.ignore();
    cout << "Enter Name: "; getline(cin, newStudent -> name);
    cout << "Enter Age: ";    cin >> newStudent -> age;
    cout << "Enter Gender: ";    cin >> newStudent -> gender;
    cout << "Enter Department: ";    cin >> newStudent -> department;
    cin.ignore();
    cout << "Enter Major: ";    getline(cin, newStudent -> major);
    cout << "Enter Year: ";    cin >> newStudent -> year;
    cin.ignore();
    cout << "Enter Province: ";    getline(cin, newStudent -> province);
    cout << "Enter Phone Number: ";    getline(cin, newStudent -> phone);
    cout << "Enter Birth (DD/MM/YY): ";    cin >> newStudent -> birth;
    cout << "Enter Email: ";    cin >> newStudent -> email;
    newStudent -> next = NULL;
}
```

```
if (rear == NULL) {  
    front = rear = newStudent;  
}else {  
    rear -> next = newStudent;  
    rear = newStudent;  
}  
SaveFile();  
cout << "\nStudent Added Successfully." << endl;  
}
```

Terminal

Enter Student ID: **123**

Enter Name: **Rak**

Enter Age: **18**

Enter Gender: **Male**

Enter Department: **AMS**

Enter Major: **Data Science**

Enter Year: **2**

Enter Province: **Phnum Penh**

Enter Phone Number: **088 7171 17**

Enter Birth (DD/MM/YY): **01/03/2006**

Enter Email: **rakkkh06@gmail.com**

Student Added Successfully.

VI. Remove Student

Code

```
//Remove student

void removeStudent () {
    if (front == NULL) {
        cout << "\nThere is no student to show..!" << endl;
        return;
    }

    string targetID;
    cout << "\nEnter Student's ID to delete: ";    cin >> targetID;

    Student* current = front;
    Student* prev = NULL;

    while (current != NULL && current -> id != targetID) {
        prev = current;
        current = current -> next;
    }

    if (current == NULL) {
        cout << "\nStudent not found" << endl;
        return;
    }

    if (prev == NULL) {
        front = current -> next;
```

```

    }else {

        prev -> next = current -> next;

    }

    if (current == rear){

        rear = prev;

    }

    delete current;

    SaveFile();

    cout << "\nStudent delete successfully." << endl;

}

```

Terminal

We added 4 students.

- Before deleted

----- Student List -----

ID	Name	Age	Gender	department	Major	Year ...
e20241363	Hin Panha	19	Male	AMS	Data Science	2 ...
e20240263	Hor Soue Hak	19	Male	AMS	Data Science	2 ...
e20240486	Kun Virak	18	Male	AMS	Data Science	2 ...
123	Rak	18	Male	AMS	Data Science	2 ...

When user select choice that wants to delete a student.

Enter Student's ID to delete: 123

Student delete successfully.

- After deleted

----- Student List -----

ID	Name	Age	Gender	department	Major	Year	...
e20241363	Hin Panha	19	Male	AMS	Data Science	2	...
e20240263	Hor Soue Hak	19	Male	AMS	Data Science	2	...
e20240486	Kun Virak	18	Male	AMS	Data Science	2	...

- Delete a student by ID.
- Search for the target record.
- Update queue links after deletion.
- Save changes to the file.

VII. Sort Student Records

 Code

```
//Sorted name A -> Z

void sortName() {

    if (front == NULL || front -> next == NULL)

        return;

    bool swapped;

    Student* current;

    do {

        swapped = false;

        current = front;

        while (current -> next != NULL) {
```

```

if (current -> name > current -> next -> name) {

    swap(current -> id, current -> next -> id);
    swap(current -> name, current-> next-> name);
    swap(current -> age, current -> next -> age);
    swap(current -> gender, current -> next -> gender);
    swap(current -> department, current -> next -> department);
    swap(current -> major, current -> next -> major);
    swap(current -> year, current -> next->year);
    swap(current -> province, current -> next -> province);
    swap(current -> phone, current -> next -> phone);
    swap(current -> birth, current -> next -> birth);
    swap(current -> email, current -> next -> email);

    swapped = true;
}

current = current -> next;
}

} while (swapped);
}

```

- Sort students alphabetically by name.
- Use Bubble Sort technique.
- Improve readability when displaying records.

VIII. Display Student

Code

```
//Display Student

void displayStudent () {
    if (front == NULL) {
        cout << "\nThere is no student...!" << endl;
        return;
    }
    sortName();

    //header
    cout << "\n\n----- Student List ----- \n\n";
    cout << left
        << setw (15) << "ID"
        << setw (20) << "Name"
        << setw (10) << "Age"
        << setw (15) << "Gender"
        << setw (15) << "department"
        << setw (20) << "Major"
        << setw (10) << "Year"
        << setw (15) << "Province"
        << setw (20) << "Phone"
        << setw (20) << "Birth"
        << setw (30) << "Email" << endl;

    Student* show = front;
```

```

while (show != NULL) {
    cout << left
        << setw(15) << show -> id
        << setw(20) << show -> name
        << setw(10) << show -> age
        << setw(15) << show -> gender
        << setw(15) << show -> department
        << setw(20) << show -> major
        << setw(10) << show -> year
        << setw(15) << show -> province
        << setw(20) << show -> phone
        << setw(20) << show -> birth
        << setw(30) << show -> email << endl;

    show = show -> next;
}
}

```

- Display all student records.
- Show information in table format using setw().
- Call the sorting function before displaying.

IX. Search Student

Code

```
//Helper function
Student* findStudent (string id){
    Student* look = front; //look point to the first node

    while (look != NULL){
        if (look -> id == id){
            return look;
        }

        look = look -> next;
    }

    return NULL;
}

//Search student
void searchStudent (){
    if (front == NULL){
        cout << "\nThere is no student." << endl;
        return;
    }

    string targetID;

    cout << "\nEnter ID student to see their information: ";    cin >>
targetID;

    Student* found = findStudent(targetID);
```

```

if (found == NULL) {

    cout << "\nStudent not found." << endl;

    return;

}

cout << "\nStudent Found!\n";

cout << "ID: " << found -> id << endl;

cout << "Name: " << found -> name << endl;

cout << "Age: " << found -> age << endl;

cout << "Gender: " << found -> gender << endl;

cout << "Department: " << found -> department << endl;

cout << "Major: " << found -> major << endl;

cout << "Year: " << found -> year << endl;

cout << "Province: " << found -> province << endl;

cout << "Phone: " << found -> phone << endl;

cout << "Birth: " << found -> birth << endl;

cout << "Email: " << found -> email << endl;

}

```

Terminal

When user select choice that wants to search for a student.

Enter ID student to see their information: **e20241363**

Student Found!

ID: e20241363

Name: Hin Panha

Age: 19

Gender: Male

Department: AMS

Major: Data Science

Year: 2

Province: Battambang

Phone: 088 9157 750

Birth: 26/12/2006

Email: panhaahinn@gmail.com

- Search a student using their ID.
- Display complete student information if found.
- Inform the user if the record does not exist.

X. Update Student Information

Code

```
//Update student's information by their ID
void updateStudent () {
    if (front == NULL) {
        cout << "\nThere is no student." << endl;
        return;
    }
    string targetID;
    cout << "\nEnter ID student to update their information: ";
    cin >> targetID;
    Student* update = findStudent(targetID);
```

```

if (update == NULL) {

    cout << "\nStudent not found." << endl;

    return;

}

int choice;

cout << "\n----- Update Menu ----- \n";

cout << "\n01. Update ID\n";

cout << "02. Update Name\n";

cout << "03. Update Age\n";

cout << "04. Update Gender\n";

cout << "05. Update Department\n";

cout << "06. Update Major\n";

cout << "07. Update Year\n";

cout << "08. Update Province\n";

cout << "09. Update Phone\n";

cout << "10. Update Birth\n";

cout << "11. Update Email\n";

cout << "\nEnter Choice: "; cin >> choice;

switch (choice) {

    case 1:

        cout << "Enter New ID: ";

        cin >> update->id;

        break;

    case 2:

        cin.ignore();

        cout << "Enter New Name: ";

```

```

        getline(cin, update->name);

        break;

case 3:

    cout << "Enter New Age: ";

    cin >> update -> age;

    break;

case 4:

    cout << "Enter New Gender: ";

    cin >> update -> gender;

    break;

case 5:

    cout << "Enter New Department: ";

    cin >> update -> department;

    break;

case 6:

    cin.ignore();

    cout << "Enter New Major: ";

    getline(cin, update -> major);

    break;

case 7:

    cout << "Enter New Year: ";

    cin >> update -> year;

    break;

```

```

    case 8:

        cin.ignore();

        cout << "Enter New Province: ";

        getline(cin, update -> province);

        break;

    case 9:

        cin.ignore();

        cout << "Enter New Phone Number: ";

        getline(cin, update -> phone);

        break;

    case 10:

        cout << "Enter New Birth (DD/MM/YYYY): ";

        cin >> update -> birth;

        break;

    case 11:

        cout << "Enter New Email: ";

        cin >> update -> email;

        break;

    default:

        cout << "\nInvalid Choice.\n";

        return;

}

SaveFile();

cout << "\nStudent information updated successfully.\n";

}

```

Terminal

Enter ID student to update their information: **e20241363**

----- Update Menu -----

- 01. Update ID
- 02. Update Name
- 03. Update Age
- 04. Update Gender
- 05. Update Department
- 06. Update Major
- 07. Update Year
- 08. Update Province
- 09. Update Phone
- 10. Update Birth
- 11. Update Email

Enter Choice: **1**

Enter New ID: **e20241111**

Student information updated successfully.

- Modify existing student information.
- Allow updating specific fields such as name, age, department, phone, and email.
- Save changes automatically.

XI. Menu Function

Code

```
//Menu
void showMenu() {
    cout <<
    "\n----- Student Management System ----- \n\n";

    cout << "1. Add Student\n";
    cout << "2. Remove Student\n";
    cout << "3. Display Students\n";
    cout << "4. Search Student\n";
    cout << "5. Update Student\n";
    cout << "6. Exit\n";
    cout << "\nEnter Choice: ";
}
```

Terminal

```
----- Student Management System -----
```

1. Add Student
2. Remove Student
3. Display Students

4. Search Student

5. Update Student

6. Exit

- Display available system options.
- Provide a user-friendly interface.

XII. Main Function

 Code

```
int main() {  
    DataIntoQueue();  
  
    int choice;  
  
    do {  
        showMenu();  
        cin >> choice;  
  
        switch (choice) {  
            case 1:  
                addStudent();  
                break;
```

```

        case 2:
            removeStudent();
            break;
        case 3:
            displayStudent();
            break;
        case 4:
            searchStudent();
            break;
        case 5:
            updateStudent();
            break;
        case 6:
            cout << "\nProgram Ended.\n";
            break;
        default:
            cout << "\nInvalid Choice!\n";
    }

} while (choice != 6);

return 0;
}

```

- Program entry point.
- Load existing data.
- Display menu repeatedly.
- Execute user-selected operations.
- Exit when the user chooses option 6.